

Report: Transcribing DNA into RNA

Rosalind Problem ID: RNA — Transcribing DNA into RNA

1. Problem Overview

A DNA string (coding strand) uses the alphabet A, C, G, T. An RNA string, however, is built from the alphabet A, C, G, U. Here, Thymine (T) is replaced with Uracil (U), where:

- A = Adenine
- C = Cytosine
- G = Guanine
- T = Thymine
- U = Uracil

Transcription is the biological process by which a DNA template strand acts as a template for an RNA strand, so that the coding strand is used to generate an RNA strand. At the level of the string, this is a simple, deterministic character substitution: every occurrence of the base Thymine (T) in the DNA string is replaced by Uracil (U) in the RNA string, while every A, C, and G is left unchanged.

Given: A DNA string t of length at most 1000 nt.

Return: The task is the transcribed RNA string u , where every T in t has been replaced by U.

Sample Dataset

```
GATGGAACCTTGACTACGTAAATT
```

Sample Output

```
GAUGGAACUUGACUACGUAAAUU
```

2. Approach

Since transcription (in this simplified string sense) only substitutes one specific character for another, and leaves the string's length, order, and every other character untouched, the problem reduces to a single-pass, single-character find-and-replace over the input string.

No biological complexity (such as strand direction, complementary base pairing, or reading frames) needs to be modeled here — only a direct character substitution $T \rightarrow U$.

The steps are:

- Read the DNA string t from the input file.
- Strip any trailing whitespace/newline characters introduced by file I/O.
- Replace every T character with U.
- Output the resulting RNA string u to a file.

3. Algorithm

Transcribe(t):

An RNA string is a string formed from the alphabet containing 'A', 'C', 'G', and 'U'. Given a DNA string t corresponding to a coding strand, its transcribed RNA string u is formed by replacing all occurrences of 'T' in t with 'U' in u .

```
1. u ← empty string
2. FOR each character c in t:
3.     IF c == 'T':
4.         append 'U' to u
5.     ELSE:
6.         append c to u
7. RETURN u
```

In words:

- Start with a blank/empty result string u .
- Go through the DNA string t one character at a time (call the current character c).
- If c is 'T', append 'U' to u instead of the original character.
- Otherwise (c is 'A', 'C', or 'G'), append c to u unchanged.
- Once every character has been checked, return the completed RNA string u .

In Python, this loop-based algorithm is expressed compactly using the built-in `str.replace()` method, which performs the same linear scan-and-substitute operation internally.

4. Complexity Analysis

- Time Complexity: $O(n)$, where n is the length of the DNA string t . Each character is visited exactly once.
- Space Complexity: $O(n)$, since a new string u of the same length as t is created (strings are immutable in Python).

Given the constraint ($n \leq 1000$), this problem is trivially fast — well under a millisecond even with the simplest implementation.

5. Python Implementation

Below is the finalized implementation. The logic is organized into four small, well-documented functions rather than one flat script, which makes it easier for any reader to follow: each function has a single, clearly named responsibility, and a `main()` function ties the steps together.

```
"""
Rosalind Problem: Transcribing DNA into RNA
-----
Task:
    Given a DNA string 't' (length at most 1000 nt), produce its
    transcribed RNA string 'u' by replacing every occurrence of
    Thymine ('T') with Uracil ('U'). All other bases (A, C, G)
    stay the same.

Input:
    A plain text file named 'rosalind_rna.txt' containing a single
    line with the DNA string.

Output:
    A plain text file named 'rosalind_rna_output.txt' containing a
    single line with the transcribed RNA string.
"""

# -----
# File names used for input and output.
# Keeping them as named constants (instead of "magic strings" scattered
```

```

# through the code) makes the script easier to read and update.
# -----
INPUT_FILE = "rosalind_rna.txt"
OUTPUT_FILE = "rosalind_rna_output.txt"

# -----
# FUNCTIONS USED IN THIS SCRIPT (4 custom functions total):
# 1. transcribe_dna_to_rna(dna_string) -> does the actual DNA->RNA conversion
# 2. read_dna_sequence(file_path) -> reads the DNA string from a file
# 3. write_rna_sequence(file_path, rna) -> writes the RNA string to a file
# 4. main() -> runs steps 1-3 in order
#
# Each one is defined with "def" below and marked with a comment
# "# FUNCTION:" right above it, so they're easy to spot.
# -----

# FUNCTION: transcribe_dna_to_rna
def transcribe_dna_to_rna(dna_string):
    """
    Transcribe a DNA sequence into an RNA sequence.

    Every 'T' (Thymine) in the DNA string is replaced with
    'U' (Uracil), which is the only difference between DNA and RNA
    at the level of a coding strand.

    Parameters
    -----
    dna_string : str
        A DNA sequence made up of the letters A, C, G, and T.

    Returns
    -----
    str
        The transcribed RNA sequence, with every 'T' replaced by 'U'.

    Example
    -----
    >>> transcribe_dna_to_rna("GATGGAACCTTGACTACGTAAATT")
    'GAUGGAACUUGACUACGUAAAUU'
    """
    return dna_string.replace("T", "U")

# FUNCTION: read_dna_sequence
def read_dna_sequence(file_path):
    """
    Read a DNA sequence from a text file.

    The file is expected to contain a single line with the DNA
    string. Leading/trailing whitespace (including the newline at
    the end of the file) is removed.

    Parameters
    -----
    file_path : str
        Path to the input file containing the DNA sequence.

    Returns
    -----
    str

```

```

        The DNA sequence as a clean string (no surrounding whitespace).
    """
    with open(file_path, "r") as input_file:
        return input_file.read().strip()

# FUNCTION: write_rna_sequence
def write_rna_sequence(file_path, rna_string):
    """
    Write an RNA sequence to a text file, followed by a newline.

    Parameters
    -----
    file_path : str
        Path to the output file to create (or overwrite).
    rna_string : str
        The RNA sequence to write to the file.
    """
    with open(file_path, "w") as output_file:
        output_file.write(rna_string + "\n")

# FUNCTION: main
def main():
    """
    Run the full workflow:
    1. Read the DNA sequence from INPUT_FILE.
    2. Transcribe it into RNA.
    3. Write the RNA sequence to OUTPUT_FILE.
    4. Print the result to the screen as well, for a quick check.
    """
    dna_string = read_dna_sequence(INPUT_FILE)
    rna_string = transcribe_dna_to_rna(dna_string)
    write_rna_sequence(OUTPUT_FILE, rna_string)

    print(f"Transcribed RNA string written to '{OUTPUT_FILE}':")
    print(rna_string)

# The "if __name__ == '__main__':" guard means main() only runs when
# this file is executed directly (e.g. `python rosalind_rna_solution.py`),
# not when it's imported as a module into another script.
if __name__ == "__main__":
    main()

```

6. Result

Input (rosalind_rna.txt): a single-line DNA string of 952 nt. First and last 60 characters shown below (the full sequence is omitted for brevity):

```

First 60 nt:
TTGGAGCGATGTTTGCCACGAATACCTTATTAGCACCAGACATACGCTGTAACAACAGAA

Last 60 nt:
GCGTCCCTCATCAGCCTGATCCGGCTGTTCAGAAACGGGTAACGCTCGTAAATACTGAT

```

Output (rosalind_rna_output.txt): the transcribed RNA string, same length (952 nt), with every T replaced by U:

```

First 60 nt:
UUGGAGCGAUGUUUGCCACGAUACCUUAUUAGCACCAGACAUACGUGUAACAACAGAA

```

Last 60 nt:

```
GCGUCCCUCAUCAGCCUGAUCCGGCUGUUCAGAAACGGGUAACGCUCGUAAAUACUGAU
```

The output matches the expected sample output format exactly (every T replaced by U, all other characters and the string length unchanged), confirming the correctness of the implementation. Rosalind accepted the submitted output as correct.

7. Skills Demonstrated

This introductory problem provided practice in several fundamental programming and bioinformatics concepts.

Python Programming

- Functions
- For loops (conceptually, underlying `str.replace()`)
- String processing
- File input/output

Algorithmic Thinking

- Designing a single-pass substitution algorithm
- Selecting an appropriate data structure
- Understanding time and space complexity

Bioinformatics

- Representing DNA and RNA sequences computationally
- Processing the transcription of DNA into RNA

Scientific Programming Practice

- Writing reusable, single-responsibility functions
- Documenting code with docstrings and comments
- Separating computational logic from input/output
- Recording and interpreting computational results

8. Conclusion

This problem demonstrates that not every bioinformatics task requires complex algorithms — sometimes biological processes translate directly into simple, well-known string operations. Here, DNA-to-RNA transcription (in its string-level abstraction) is nothing more than a single-character replacement, solvable in $O(n)$ time with a one-line Python expression.